

Notas completas de Streamlit

Índice

1. Estructura de proyectos en Streamlit
2. Sidebar y elementos de selección
3. Layout: columnas y expanders
4. Modelo de ejecución de Streamlit
5. Uso de `with` con contenedores
6. `st.session_state`
7. API de chat: `st.chat_message` y `st.chat_input`
8. `@st.cache_resource`

1. Estructura de proyectos en Streamlit

1.1. App pequeña

```
mi_app/
├─ app.py           # Punto de entrada de la app
├─ requirements.txt # Dependencias (opcional)
└─ data/           # Datos de entrada/salida (si los hay)
```

- Archivo principal: usualmente `app.py` o `streamlit_app.py` (lo ejecutas con `streamlit run app.py`).
- Carpeta `data/` para no mezclar datos con código.

1.2. App mediana con multipágina

```
mi_app/
├─ streamlit_app.py # Punto de entrada
├─ pages/           # Otras páginas de la app
│   └─ 1_📊_Dashboard.py
│   └─ 2_⚙️_Configuración.py
├─ data/
│   └─ input/
│   └─ output/
├─ src/             # Lógica de negocio / utilidades
│   └─ __init__.py
│   └─ utils.py
│   └─ modelo.py
├─ .streamlit/      # Configuración de la app
│   └─ config.toml
└─ requirements.txt
```

Ideas clave:

- Todo `.py` dentro de `pages/` se convierte en una página en el sidebar.
- `src/` o `utils/` para funciones reutilizables (carga de datos, modelos, etc.).
- `.streamlit/config.toml` para tema, ancho de página, etc.

2. Sidebar y elementos de selección

2.1 Sidebar básico

Ejemplo normal (sin `with`):

```
import streamlit as st

st.title("Ejemplo sidebar - normal")

st.sidebar.title("Menú lateral")
pagina = st.sidebar.radio("Ir a:", ["Inicio", "Análisis de datos", "Acerca de"])
filtro = st.sidebar.slider("Filtro:", 0, 10, 5)

st.write("Página:", pagina)
st.write("Filtro:", filtro)
```

Ejemplo con with st.sidebar:

```
import streamlit as st

st.title("Ejemplo sidebar - con with")

with st.sidebar:
    st.title("Menú lateral")
    pagina = st.radio("Ir a:", ["Inicio", "Análisis de datos", "Acerca de"])
    filtro = st.slider("Filtro:", 0, 10, 5)

st.write("Página:", pagina)
st.write("Filtro:", filtro)
```

2.2 Selectbox (desplegable, una opción)

```
import streamlit as st

st.title("Selectbox y multiselect")

opcion = st.selectbox(
    "Selecciona una fruta:",
    ["Manzana", "Banana", "Naranja"]
)

st.write("Elegiste:", opcion)
```

2.3 Multiselect (desplegable, varias opciones)

```
frutas = st.multiselect(
    "Elige varias frutas:",
    ["Manzana", "Banana", "Naranja", "Mango"],
    default=["Manzana"]
)

st.write("Frutas seleccionadas:", frutas)
```

2.4 Radio (una opción visible)

```
modo = st.radio(
    "Modo de visualización:",
    ["Simple", "Avanzado"]
)

st.write("Modo actual:", modo)
```

Regla mental: pocas opciones → radio ; muchas → selectbox .

3. Layout: columnas y expanders

3.1 Columnas

Ejemplo normal (sin `with`):

```
import streamlit as st

st.title("Columnas - ejemplo normal")

col1, col2 = st.columns(2)

col1.header("Columna 1")
col1.write("Texto en la columna 1")
col2.header("Columna 2")
col2.write("Texto en la columna 2")
```

Ejemplo con `with`:

```
import streamlit as st

st.title("Columnas - con with")

col1, col2 = st.columns(2)

with col1:
    st.header("Columna 1")
    st.write("Texto en la columna 1")

with col2:
    st.header("Columna 2")
    st.write("Texto en la columna 2")
```

3.2 Expander

Ejemplo normal:

```
import streamlit as st

st.title("Expander - normal")

exp = st.expander("Mostrar opciones avanzadas")
exp.write("Aquí van las opciones avanzadas...")
exp.checkbox("Activar modo experto", key="experto")
```

Ejemplo con `with`:

```
import streamlit as st

st.title("Expander - con with")

with st.expander("Mostrar opciones avanzadas"):
    st.write("Aquí van las opciones avanzadas...")
    st.checkbox("Activar modo experto", key="experto")
```

4. Modelo de ejecución de Streamlit

- El script se ejecuta **de arriba a abajo** como un archivo de Python normal.
- Se vuelve a ejecutar completo cuando:
 - Cambias el código y guardas.

- El usuario interactúa con un widget (slider, botón, etc.).

No hay un `while True`; el "loop" es:

interacción → rerun completo del script.

Por eso:

- Variables normales se reinician en cada ejecución.
- Para recordar cosas se usa `st.session_state`.
- Para recursos pesados se usa caching (`@st.cache_data`, `@st.cache_resource`).

5. Uso de `with` con contenedores

El `with` se usa para **contenedores**:

- `st.sidebar`
- `st.columns` (`col1, col2, ...`)
- `st.expander`
- `st.tabs`
- `st.container`
- `st.chat_message`

Ejemplo general:

```
with st.sidebar:
    st.write("Todo esto va al sidebar")

col1, col2 = st.columns(2)
with col1:
    st.write("Contenido de la columna 1")
with col2:
    st.write("Contenido de la columna 2")
```

El `with` NO cambia el orden de ejecución, solo agrupa visualmente dentro del contenedor.

6. `st.session_state`

¿Qué es?

`st.session_state` es un diccionario especial que **sobrevive entre ejecuciones** del script en la misma sesión del usuario.

Se usa para recordar:

- Contadores
- Configuraciones seleccionadas
- Historial de chat
- Paso actual de un flujo, etc.

Ejemplo 1: contador

```
import streamlit as st

st.title("Ejemplo de st.session_state")

if "contador" not in st.session_state:
    st.session_state.contador = 0

if st.button("Sumar 1"):
    st.session_state.contador += 1

st.write("Valor del contador:", st.session_state.contador)
```

Ejemplo 2: guardar nombre introducido

```
import streamlit as st

if "nombre" not in st.session_state:
    st.session_state.nombre = ""

nombre = st.text_input("Escribe tu nombre:", value=st.session_state.nombre)

if nombre:
    st.session_state.nombre = nombre

st.write("Nombre almacenado:", st.session_state.nombre)
```

7. API de chat: `st.chat_message` y `st.chat_input`

7.1 `st.chat_message(role)`

Crea un mensaje de chat con un rol:

```
import streamlit as st

st.title("Ejemplo simple de chat_message")

with st.chat_message("assistant"):
    st.write("Hola, soy tu asistente en Streamlit.")
```

Roles típicos: "user", "assistant" (puedes usar otros).

7.2 `st.chat_input(prompt)`

Caja de texto de chat fija abajo; devuelve el texto enviado:

```
import streamlit as st

st.title("Ejemplo de chat_input")

mensaje = st.chat_input("Escribe algo:")

if mensaje:
    st.write("Escribiste:", mensaje)
```

7.3 Chat "eco" con historial (combina todo)

```
import streamlit as st

st.title("Chat eco con historial")

# Inicializar historial
if "mensajes" not in st.session_state:
    st.session_state.mensajes = []

# Mostrar historial completo
for msg in st.session_state.mensajes:
    with st.chat_message(msg["role"]):
        st.write(msg["content"])

# Entrada de chat
texto = st.chat_input("Escribe un mensaje:")

if texto:
    # Mensaje del usuario
    st.session_state.mensajes.append(
        {"role": "user", "content": texto}
    )

    # Respuesta sencilla (eco)
    respuesta = f"Recibí: {texto}"

    # Mensaje del assistant
    st.session_state.mensajes.append(
        {"role": "assistant", "content": respuesta}
    )

    # Mostrar solo la última respuesta del assistant
    with st.chat_message("assistant"):
        st.write(respuesta)
```

8. @st.cache_resource

¿Qué es?

Decorador para cachear **recursos de larga vida**, típicamente:

- Modelos de ML.
- Conexiones a bases de datos.
- Clientes de APIs.

Se diferencia de:

- `@st.cache_data`: cachea resultados de funciones de datos.
- `@st.cache_resource`: cachea objetos / recursos.

Ejemplo: modelo pesado

```
import streamlit as st
import time

@st.cache_resource
def cargar_modelo_pesado():
    # Simula una carga muy lenta
    time.sleep(5)
    return "modelo_pesado_listo"

st.title("Ejemplo de @st.cache_resource")

st.write("Cargando modelo...")
modelo = cargar_modelo_pesado()
st.write("Modelo:", modelo)
st.write("Si interactúas de nuevo, ya no espera 5 segundos.")
```